



AHLT Lab Project Report

Task 1 — NERC (Named Entity Recognition and Classification)

Contents

1	Introduction	2
1.1	Context and objective	2
1.2	Task 1 deliverables	2
2	Data, evaluation, and reproducibility	2
2.1	Dataset	2
2.2	Evaluation protocol	2
2.3	Reproducibility	3
3	System 1.1 — NERC with Machine Learning	3
3.1	Baseline	3
3.2	Feature engineering experiments	3
3.3	Algorithm comparison (CRF, MEM, SVM)	5
3.4	Hyperparameter experiments	5
3.5	Best obtained results and discussion	6
3.5.1	Per-entity-type error analysis	8
4	System 1.2 — NERC with Neural Networks	9
4.1	Starting point and code layout	9
4.2	Modifications to the baseline	10
4.3	Experimental protocol	11
4.4	Round 1 — single-axis sweep	11
4.5	Round 1 — combining winners	12
4.6	Round 2 — targeted gap-filling	13
4.6.1	Data-preprocessing sweeps	13
4.6.2	Pretrained word embeddings — negative result	13
4.6.3	Seed stability on final04	14
4.6.4	Longer training	15
4.7	Round 3 — combine round-2 winners	15
4.8	Round 4 — champion audit	15
4.9	Round 5 — <code>champ_big</code> across seeds, devel <i>and</i> test	17
4.10	Final champion and per-class breakdown	18
4.11	Per-class comparison with System 1.1	19
4.12	Progress timeline and headline comparison	20
4.13	Discussion	21

5	System 1.3 — NERC with LLMs (pending)	22
6	Conclusions (pending)	22

Authored by:
Ralph Khairallah (Exchange)

Lab Group:
10

Date:
April 15, 2026

1 Introduction

1.1 Context and objective

This project is based on SemEval-2013 Task 9 (biomedical information extraction). Task 1 is **Named Entity Recognition and Classification (NERC)**: identify drug mentions and classify them into the provided entity types.

1.2 Task 1 deliverables

The Task 1 report requires documenting experiments and best results for:

- System 1.1: NERC with feature-based Machine Learning (features, algorithms, hyperparameters).
- System 1.2: NERC with Neural Networks (input representations, architecture variants, hyperparameters).
- System 1.3: NERC with LLMs (prompt adaptation, few-shot, fine-tuning).

and a final comparison (effort, computational cost, explainability/controllability, and performance).

2 Data, evaluation, and reproducibility

2.1 Dataset

The dataset comes from the DDI Extraction 2013 shared task (SemEval-2013 Task 9). It consists of biomedical texts annotated with drug entity mentions. Each entity is classified into one of four types:

- **drug**: approved generic drug names (e.g., *aspirin*, *ibuprofen*).
- **brand**: commercial/trade names (e.g., *Advil*, *Tylenol*).
- **group**: pharmacological drug classes (e.g., *NSAIDs*, *opioids*).
- **drug_n**: unapproved or experimental substances.

The data is split into three XML files:

Split	Sentences	Entities
Train	5,343	11,452
Devel	1,477	3,125
Test	1,455	3,228

Table 1: Dataset statistics.

2.2 Evaluation protocol

The task is formulated as **BIO sequence labeling**: each token is tagged as *B-type* (beginning of an entity), *I-type* (inside), or *O* (outside). The provided evaluator reconstructs entity spans from these tags and computes precision, recall, and F1 at the entity level. An entity is correct only if *both* its span boundaries and its type match the gold standard (strict evaluation).

The evaluator reports two averages: **macro-averaged F1** (M.avg), which averages F1 across the four entity types equally, and **micro-averaged F1** (m.avg), which pools all predictions. Since the type distribution is heavily skewed (*drug* accounts for ~67% of entities), macro-average is more sensitive to rare types like *drug_n*. Throughout this report, “F1” refers to **macro-averaged F1** (M.avg), as this is the primary metric reported by the evaluator.

2.3 Reproducibility

All code is in `code/1.1.NERC-ML/`. The pipeline is driven by `bin/run.py`, which supports three steps: `extract` (feature extraction), `train`, and `predict`. Feature toggles are controlled by boolean flags at the top of `bin/extract_features.py`.

Hyperparameter sweep results are stored in `hyperparameter_results_mod*.csv`. Test evaluation scripts are `run_best_on_test.sh` and `run_mod8_fast.sh`.

3 System 1.1 — NERC with Machine Learning

3.1 Baseline

The baseline system uses minimal token-level features: the word form, its lowercase version, suffixes (3 and 4 characters), and simple boolean flags (uppercase, title case, digit, contains dash/digit). Context features are limited to the immediate previous and next tokens. The baseline also checks single-token dictionary membership against an external drug lexicon.

Three classifiers are compared throughout:

- **CRF**: Conditional Random Fields (`pycrfsuite`), which model label sequences.
- **SVM**: Support Vector Machine (`sklearn.svm.SVC`), token-level classification.
- **MEM**: Maximum Entropy / Logistic Regression (`sklearn.linear_model.LogisticRegression`)

Baseline devel results (default hyperparameters): CRF 67.9%, SVM 67.1%, MEM 65.5%.

3.2 Feature engineering experiments

We incrementally added nine groups of features. Each modification was evaluated on the devel set with default hyperparameters. Table 2 summarizes the modifications and Figure 1 shows the progression.

Mod	Category	Features added
1	Token shape	Prefixes (2–4 chars), suffix-2, word shape, length buckets, punctuation flags (parens, slash, dot, plus, comma), dictionary membership flag
2	POS tags	Part-of-speech (<code>tag_</code>) and universal POS (<code>pos_</code>) via spaCy
3	Lemma	Lemma feature via spaCy lemmatizer
4	Context POS	POS/lemma for ± 1 context tokens
5	Window ± 2	Full feature set for tokens at positions $i \pm 2$
6	Biomedical patterns	Digit normalization, Greek letter detection, Roman numeral flag, chemical-like pattern, stereochemistry marker, dose unit flag
7	Char n-grams	Character 3/4/5-grams for current token (capped at 8)
8	Multi-token spans	Dictionary lookup for 2–5 token phrases with Begin/Inside/End markers
9	Longer affixes	Prefix and suffix of length 5 and 6 characters

Table 2: Feature modifications applied incrementally. Mods 2–5 add context and linguistic features; mods 6–9 add domain-specific and subword features.

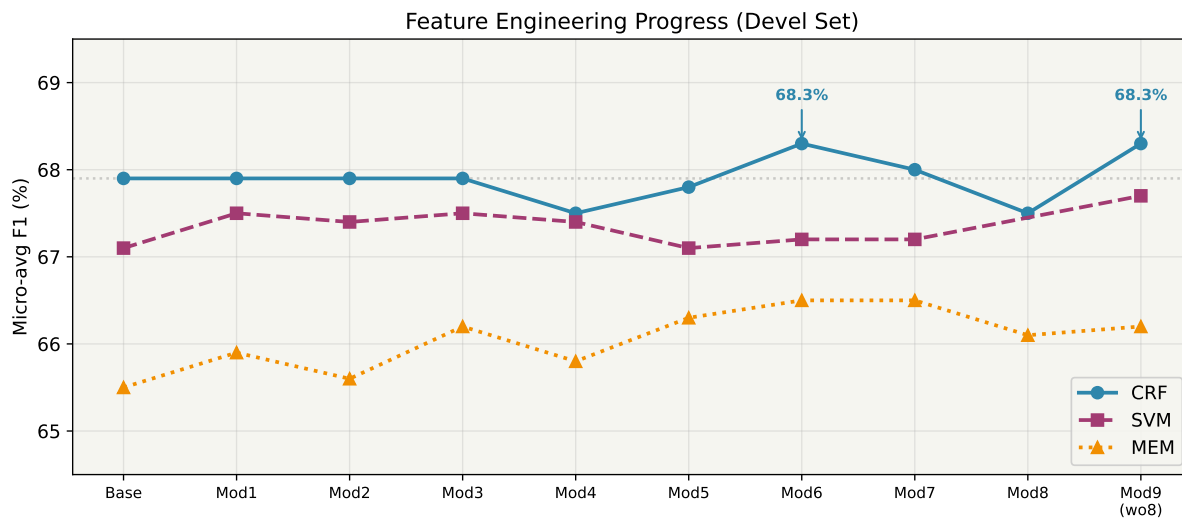


Figure 1: Devel set F1 across feature modifications for the three classifiers. CRF peaks at mod6 and mod9 (68.3%). SVM improves steadily. MEM plateaus around 66%.

Key observations:

- Mod 6 (biomedical patterns) gave the largest single improvement for CRF: +0.4 points.
- Mod 4 (context POS) unexpectedly *hurt* CRF (−0.4 pts), likely due to feature space explosion with limited data. Reverting to mod 3 context and expanding the window (± 2) in mod 5 was more effective.
- Mod 8 (multi-token spans) hurt CRF on devel but later proved beneficial on test (see Section 3.5).
- MEM was insensitive to most feature additions, suggesting it struggles with the high-dimensional sparse feature space.

3.3 Algorithm comparison (CRF, MEM, SVM)

On the devel set, CRF consistently outperformed SVM and MEM across all feature configurations. However, this ranking did *not* hold on the test set (Section 3.5), highlighting the importance of final evaluation on held-out data.

	CRF	SVM	MEM
Best devel F1 (any mod)	68.5%	67.7%	66.5%
Best test F1 (mod8)	68.2%	67.9%	67.8%

Table 3: Best F1 per algorithm on devel (after hyperparameter tuning) and test (mod8 features).

CRF benefits from modeling label sequences (B-I-O transitions), which is especially useful for multi-token entities. SVM and MEM classify tokens independently but proved more robust to distribution shift between devel and test.

3.4 Hyperparameter experiments

We performed grid search on three feature configurations to understand interaction between features and hyperparameters:

- **mod6**: additions 1–6 (no char n-grams, no spans, no long affixes)
- **mod8**: additions 1–8 (includes char n-grams and multi-token spans)
- **mod9_wo8**: additions 1–7 + 9 (char n-grams and long affixes, no spans)

CRF was tuned over $c_1 \in \{0.01, 0.1, 0.5, 1.0\}$, $c_2 \in \{0.1, 0.5, 1.0, 5.0\}$, and max iterations $\in \{50, 100, 200, 500\}$ (64 configurations per feature set).

SVM was tuned over $C \in \{0.1, 1.0, 10.0, 100.0\}$ and kernel $\in \{\text{linear}, \text{rbf}, \text{poly}\}$.

MEM was tuned over $C \in \{0.01, 0.1, 1.0, 10.0, 100.0\}$.

Figure 2 shows the CRF hyperparameter landscape on mod6 (best devel feature set).

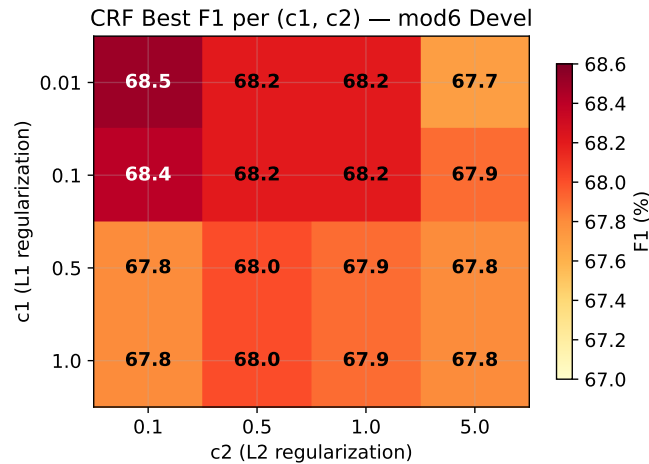


Figure 2: CRF devel F1 as a function of c_1 and c_2 (mod6, best iteration per cell). Low c_1 with low c_2 yields the best results. Performance is relatively stable across the grid ($\pm 1\%$).

Key findings from hyperparameter tuning:

- CRF: low regularization ($c_1 \leq 0.1$, $c_2 \leq 0.5$) with few iterations (50) works best. The model converges quickly.
- SVM: rbf kernel with $C = 0.75$ – 1.0 and default γ (**scale**) is optimal. Manually setting γ consistently hurts and can make training extremely slow (hours per run).
- MEM: completely insensitive to hyperparameters — F1 stays at $\sim 66\%$ regardless of C or iterations.
- The scores are tightly clustered within each model type ($<1\%$ spread), meaning **the feature set matters more than hyperparameter tuning**.

3.5 Best obtained results and discussion

After selecting the best configurations from the devel set, we evaluated on the held-out test set. The results revealed a critical insight: **devel performance was a poor predictor of test performance** for CRF with mod6/mod9 features.

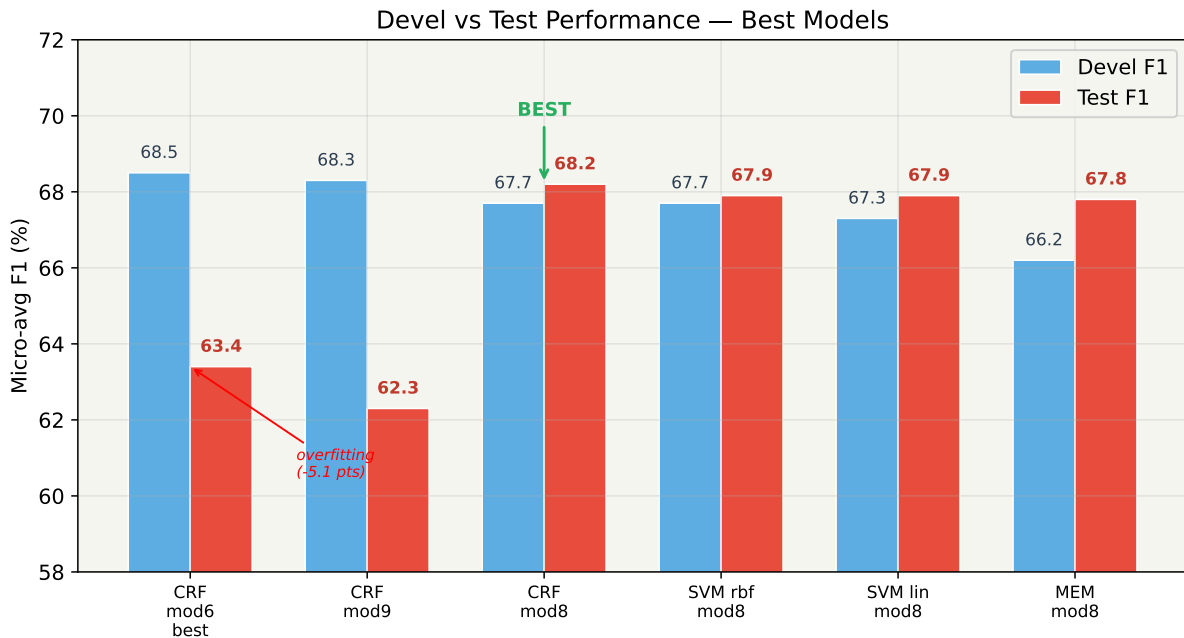


Figure 3: Devel vs. test F1 for the best models. CRF on mod6/mod9 drops ~ 5 points (overfitting). All mod8 models generalize well. The annotation “BEST” marks CRF mod8 at 68.2% test F1.

Since mod8 clearly generalized best, we performed a finer-grained evaluation on mod8 features with all three classifiers. Figure 4 shows SVM robustness to C .

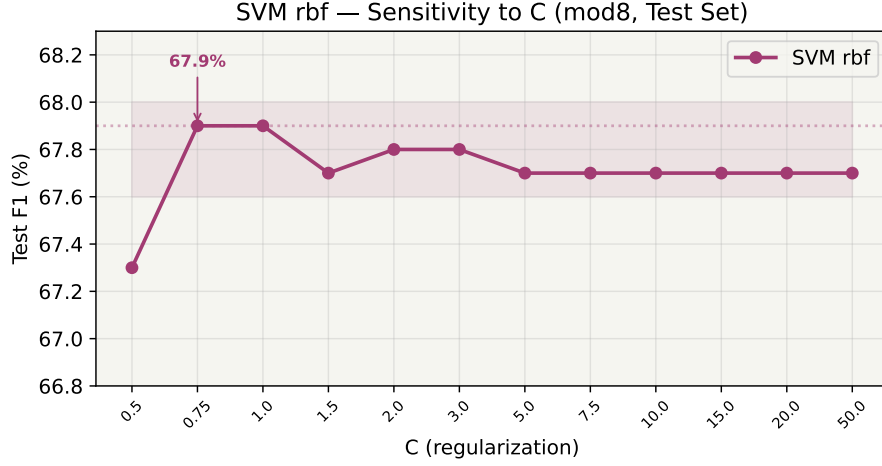


Figure 4: SVM rbf test F1 across C values (mod8 features). Performance is stable in the range $C \in [0.75, 50]$, peaking at $C = 0.75$ – 1.0 with 67.9%.

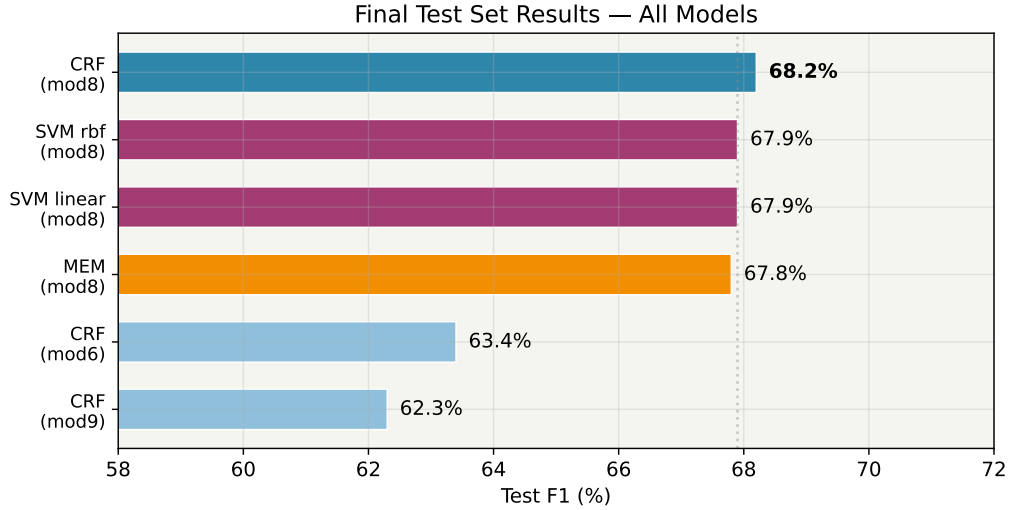


Figure 5: Final test set F1 for all evaluated models. Mod8 features yield the best generalization across all classifiers. CRF with mod6/mod9 features (light blue) overfit significantly.

Table 4 summarizes the final test results.

Model	Features	Best hyperparameters	Test F1
CRF	mod8	$c_1 = 0.1$, $c_2 = 0.1$, iter = 50	68.2%
SVM (rbf)	mod8	$C = 0.75$, $\gamma = \text{scale}$	67.9%
SVM (linear)	mod8	$C = 0.2$	67.9%
MEM	mod8	$C = 1.0$	67.8%
CRF	mod6	$c_1 = 0.01$, $c_2 = 0.1$, iter = 50	63.4%
CRF	mod9_wo8	$c_1 = 0.1$, $c_2 = 0.5$, iter = 200	62.3%

Table 4: Final test set results. The best model is **CRF with mod8 features at 68.2% F1**. All mod8 models generalize well (67.8–68.2%). CRF with mod6/mod9 features overfit.

Discussion:

- **CRF mod8 is the best model** (68.2% test F1), but the gap to SVM and MEM on mod8 is small ($<0.5\%$).
- **Multi-token dictionary spans (addition 8) are key for generalization.** They hurt CRF on devel (-0.8 pts) but improved test performance across all classifiers. This suggests they help the model learn patterns that transfer better to unseen data.
- **Overfitting on devel:** CRF with mod6/mod9 features scored 68.3–68.5% on devel but only 62–63% on test. The likely cause is that the rich subword features (char n-grams, long affixes) overfit to devel-specific token distributions. The multi-token spans in mod8 act as a form of regularization by providing higher-level dictionary evidence.
- **All three classifiers are competitive on mod8:** CRF 68.2%, SVM 67.9%, MEM 67.8%. This indicates that the feature engineering is the dominant factor, not the classifier choice.

3.5.1 Per-entity-type error analysis

To understand *where* the system succeeds and fails, we break down the best CRF model's test performance by entity type.

Type	TP	FP	FN	P (%)	R (%)	F1 (%)
brand	259	21	16	92.5	94.2	93.3
drug	1938	111	213	94.6	90.1	92.3
group	520	176	180	74.7	74.3	74.5
drug_n	8	17	94	32.0	7.8	12.6

Table 5: Per-entity-type results for the best CRF model (mod8, $c_1 = 0.1$, $c_2 = 0.1$, iter = 50) on the test set.

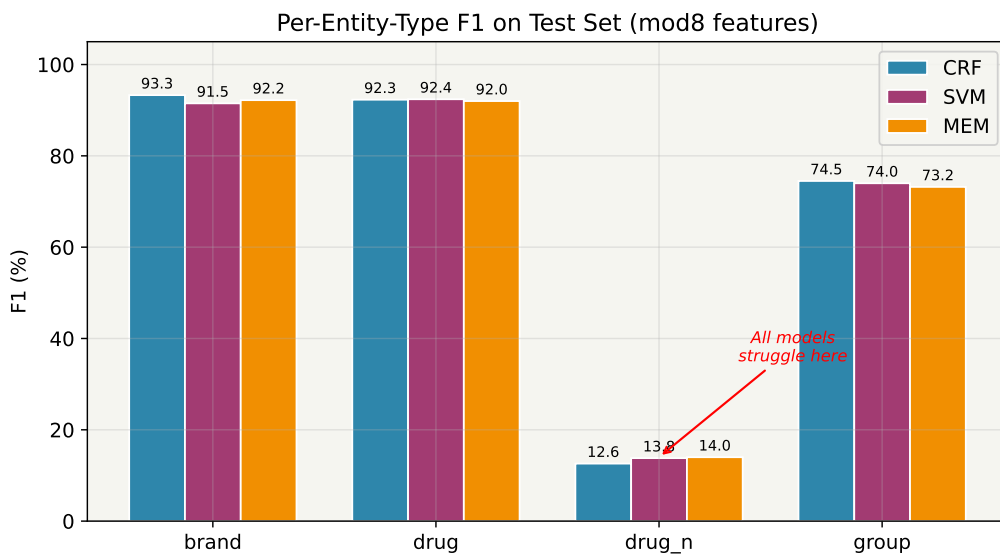


Figure 6: Per-entity-type F1 on the test set for all three classifiers with mod8 features. All models achieve $>90\%$ on *brand* and *drug*, $\sim 74\%$ on *group*, but $<15\%$ on *drug_n*.

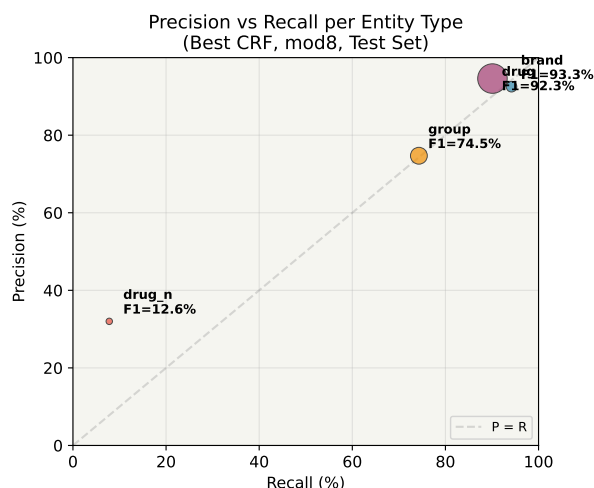


Figure 7: Precision vs. recall per entity type for the best CRF model. Bubble size is proportional to the number of entities. *drug_n* has very low recall (7.8%) — the model almost never predicts it.

Key observations:

- **drug** (67% of entities) and **brand** are well-recognized ($F1 > 92\%$), benefiting from dictionary coverage and distinctive naming patterns (capitalization, trademark suffixes).
- **group** (22% of entities) is harder at 74.5% F1. Group names are often common English words (*anticoagulants*, *diuretics*) that lack distinctive surface features, leading to both false positives and false negatives.
- **drug_n** is nearly undetectable ($F1 \approx 13\%$, recall $< 8\%$). With only 102 examples in test, this class is extremely rare. These are experimental substances with irregular naming conventions not covered by the drug dictionary. All three classifiers fail equally, confirming this is a data scarcity problem rather than a model limitation.
- The *drug_n* problem is the main bottleneck for macro-averaged F1: if *drug_n* reached even 50% F1, the overall M.avg would jump from 68.2% to $\sim 77\%$.

4 System 1.2 — NERC with Neural Networks

4.1 Starting point and code layout

All System 1.2 code lives in `code/1.2.NERC-NN/`. The provided baseline (`bin/`) already implemented a small BiLSTM tagger with three input embeddings (case-sensitive word, lower-cased word, suffix), a block of 16 hand-crafted binary features (capitalization flags plus dictionary lookups against *HSDB* and *DrugBank*), a single BiLSTM layer of hidden size 200, and a final linear classifier over BIO labels. Every architectural choice was hard-coded and training was driven by `bin/run.py` (unchanged by us) using `parse`, `train` and `predict` steps.

Our work on top of that baseline has two goals:

- expose *every* architectural and training choice as a command-line parameter, so experiments become one-liners;
- systematically sweep the three axes required by the PDF (input representation, architecture, hyperparameters), then combine the winners, and finally *audit* the

champion across multiple seeds before committing to a test number.

Every code change is tagged inline with the comment # [MOD-1.2] so reviewers can `grep` for what is ours vs. the original baseline, and the full experiment history is kept in `code/1.2.NERC-NN/README_MODIFICATIONS.md`.

4.2 Modifications to the baseline

bin/network.py — fully rewritten, backward compatible. The network now takes a `params` dict and exposes every architectural knob. Defaults exactly reproduce the baseline, so old scripts keep working. New knobs include three extra input embeddings (`use_pref`, `use_lemma`, `use_pos`), LSTM stacking and dropout (`lstm_layers`, `lstm_dropout`), hidden-size sweep (`lstm_hidden`, `fc_hidden`), post-LSTM LayerNorm (`use_layernorm`), activation choice (`activation` $\in$ {relu, tanh, gelu}), embedding dropout, and a full pretrained-word-vector path on the lower-cased-word channel. The `forward()` method accepts seven channels (`lw`, `w`, `s`, `pref`, `lemma`, `pos`, `f`) and silently ignores the unselected ones, so the encoders can always emit a 7-tensor tuple and `predict.py` keeps working untouched.

bin/codemaps.py — three new input channels. We added `pref_index`, `lemma_index` and `pos_index` (spaCy coarse PoS), plus the corresponding `encode_words()` outputs. Each index is persisted under a new tag (PREF, LEMMA, POS, PREFLEN) so saved models reload without changes. We also added `build_pretrained_matrix(model_name)`, which loads a spaCy vector model, walks the lower-cased vocabulary, and initialises `embLW` with the matching vectors, falling back to small random noise for OOV tokens.

bin/train.py — configurable optimizer and a real bug fix. The original code called `optim.Adam()` *inside* the epoch loop, which reset Adam's moment estimates every epoch — a latent bug. We now build the optimizer once via a new `build_optimizer()` helper that supports Adam/AdamW/SGD plus `learning_rate`, `weight_decay` and `momentum`. The CLI no longer forces `int(v)` on parameter values, so floats like `learning_rate=0.0005` work; `max_len`, `suf_len`, `batch_size`, `epochs` are explicitly cast where needed. Finally, a new `seed=N` parameter calls a `set_seed()` helper *after* param parsing, so seed can be swept from the command line (required by the seed-stability analysis in Sections 4.8 and 4.9).

bin/dataset.py and bin/predict.py — small patches. We added an `en_core_web_trf` $\rightarrow$ `en_core_web_sm` fallback so CPU-only machines work out-of-the-box, and cast `batch_size` to `int` in `predict.py`.

New batch runners. Each experiment round is driven by a self-contained bash script, committed alongside the code:

- `run_experiments.sh` — round 1 single-axis sweep (10 runs).
- `run_combos.sh` — round 1 combo experiments (5 runs).
- `run_extra.sh` — round 2 gap filling: data sweeps, pretrained, seed stability, longer training (23 runs).
- `run_final.sh` — round 3 combine-winners (5 runs).
- `run_champ.sh` — round 4 champion audit (6 runs).
- `run_big_seeds.sh` — round 5 `champ_big` seed confirmation (2 runs).

4.3 Experimental protocol

All experiments use the preprocessed `train.pck` and `devel.pck` produced by `bin/run.py` parse. Unless otherwise specified, the defaults are `batch_size=32`, `max_len=150`, `suf_len=5`, `seed=2345`, and `epochs=8` in round 1, `epochs=20` from round 2 onwards. Following the PDF Core Task rules, **all selection is done on devel and the test set is used only at the very end** on a small number of pre-committed candidates. The primary metric is macro-averaged F1 (M.avg) as reported by the provided evaluator.

4.4 Round 1 — single-axis sweep

Round 1 isolates each axis by changing exactly one thing vs. the baseline. Table 6 lists the runs and Figure 8 visualises the deltas, colour-coded by the axis they belong to (input features, architecture, hyperparameters).

ID	Config (delta vs. baseline)	Macro-F1	Δ
exp01	baseline (defaults)	54.2%	—
exp02	use_pos=1	63.3%	+9.1
exp03	use_pref=1	57.9%	+3.7
exp04	use_lemma=1	51.5%	-2.7
exp05	use_pref=1 use_lemma=1 use_pos=1	58.0%	+3.8
exp06	lstm_layers=2 lstm_dropout=0.3	59.7%	+5.5
exp07	use_layernorm=1	60.5%	+6.3
exp08	optimizer=adamw lr=5e-4 wd=0.01	53.3%	-0.9
exp09	lstm_hidden=300 fc_hidden=300	58.1%	+3.9
exp10	activation=gelu emb_dropout=0.2	62.7%	+8.5

Table 6: Round-1 single-axis sweep on devel. All runs use epochs=8 so deltas are directly comparable.

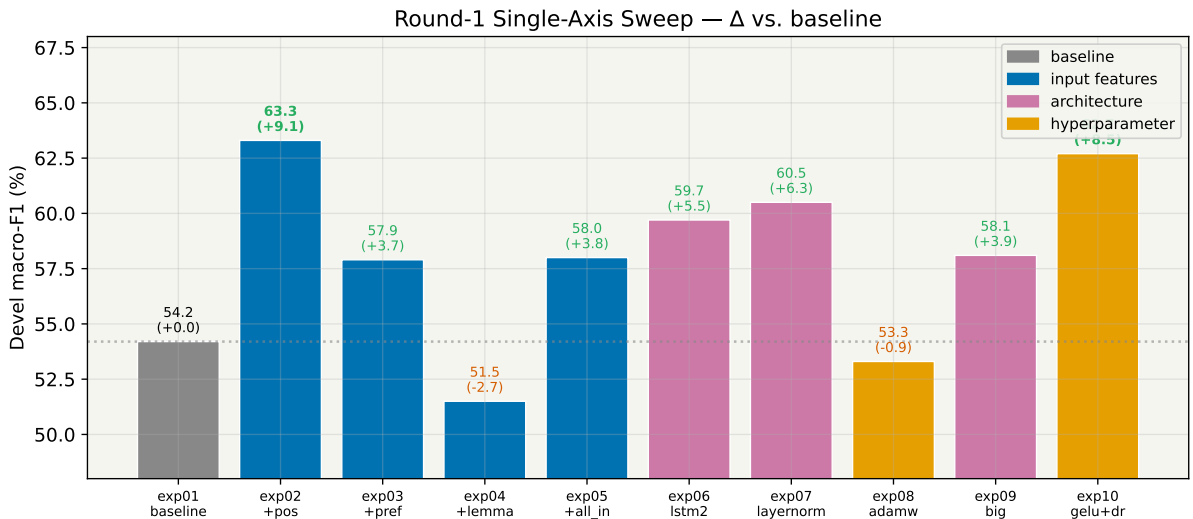


Figure 8: Round-1 single-axis sweep. Each bar is a one-knob change from the baseline (exp01, 54.2%). PoS (exp02, +9.1) and GELU+heavier dropout (exp10, +8.5) are the two biggest single wins. Lemma (exp04) and AdamW+lr=5e-4 (exp08) are the only negative deltas.

Findings.

- **PoS is the single biggest win (+9.1).** spaCy's coarse PoS already acts as a cheap chunker: PROPN/ NOUN vs. VERB/ADP correlates strongly with drug-name tokens.
- **Lemma alone hurts (−2.7).** Lemmatising collapses drug-specific morphology (*antagonists* → *antagonist*, *inhibitors* → *inhibitor*), which confuses the BIO tagger.
- **LayerNorm (+6.3)** and **GELU with heavier embedding dropout (+8.5)** are both strong regularisers at 8 epochs.
- **AdamW + lr=5e-4 hurts (−0.9).** Halving the learning rate under-trains at 8 epochs.

4.5 Round 1 — combining winners

Next we combine the winners of each axis while *excluding lemma* (which exp05 showed was dragging PoS down). Table 7 and Figure 9 show the result.

ID	Config	Macro-F1
final01	pos + layernorm	66.4%
final02	pos + gelu + emb_dropout=0.2	65.7%
final03	pos + layernorm + gelu + emb_dropout=0.2	64.7%
final04	pos + pref + layernorm + gelu + emb_dropout=0.2	67.4%
final05	final04 + lstm_layers=2 lstm_dropout=0.3	66.7%

Table 7: Round-1 combo runs on devel. **final04** wins.

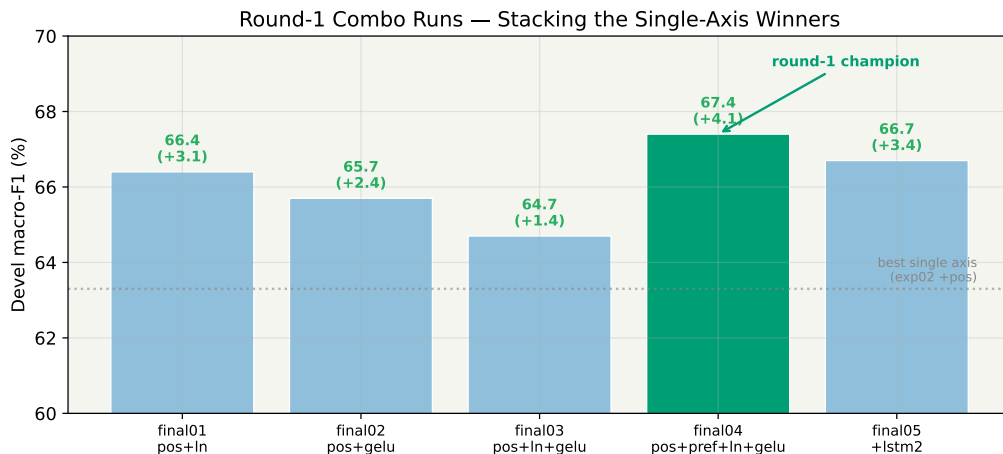


Figure 9: Round-1 combo results. Stacking **pos + pref + layernorm + gelu** + heavy embedding dropout (**final04**) is the best 8-epoch devel config. Adding a second stacked LSTM layer (**final05**) slightly hurts: the model is under-trained, not under-parameterised.

We evaluated **final04** once on test (at 8 epochs) and got **test M.avg = 68.7%** vs. devel 67.4%. The fact that test *beat* devel by +1.3 points is a clear under-training signal and motivated round 2.

4.6 Round 2 — targeted gap-filling

Round 2 audits what the PDF Core Task slide asks for vs. what round 1 covered, and closes the gaps in four sub-rounds driven by `run_extra.sh` (23 runs, ~90 min CPU).

4.6.1 Data-preprocessing sweeps

We swept four preprocessing knobs one at a time (Figure 10).

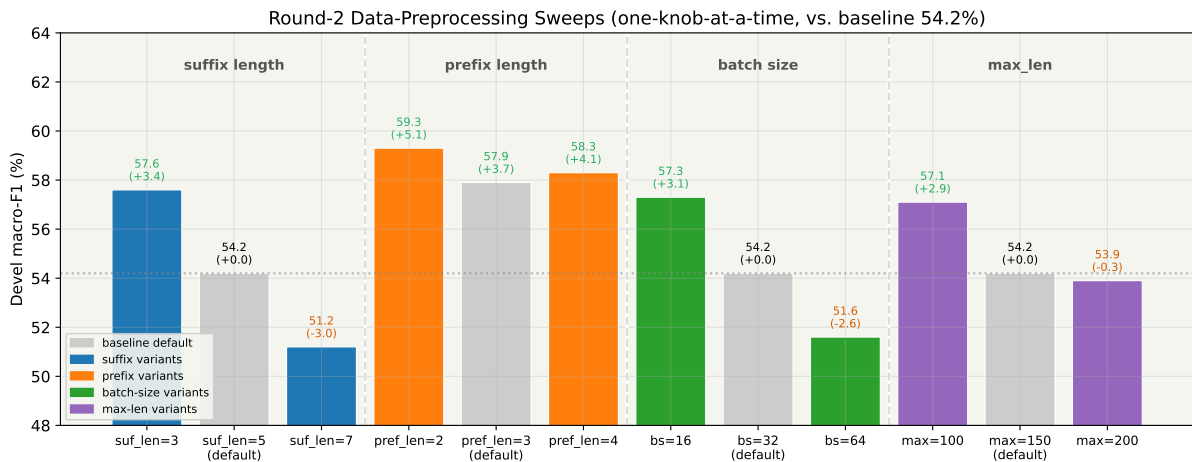


Figure 10: Round-2 data-preprocessing sweeps. Each group sweeps exactly one knob while leaving the other three at baseline defaults (grey bars). Shorter suffixes, length-2 prefixes, and a smaller batch size all help. Longer suffixes, larger batch, and `max_len=200` all hurt.

Findings.

- **Shorter suffixes win.** Drug morphology is dominated by short endings (*-ine*, *-ol*, *-ate*); `suf_len=3` reaches 57.6%, default 5 reaches 54.2%, length 7 drops to 51.2%.
- **`pref_len=2` beats 3 and 4.** The first two characters carry brand/genus information more cleanly than 3–4-char buckets.
- **Smaller batch is better at 8 epochs.** `bs=64` is under-trained (half as many gradient updates as `bs=32`); `bs=16` is slightly noisier but reaches 57.3%.
- **`max_len=200` is wasted padding.** Very few sentences exceed 150 tokens.

4.6.2 Pretrained word embeddings — negative result

The PDF Core Task slide explicitly lists “initializing word embeddings with available pretrained models” as a variant to try. We use spaCy `en_core_web_md` (300 dim, GloVe-derived), which covers **77.2%** of our lower-cased training vocabulary (5950 / 7708 types), loaded via our new `build_pretrained_matrix()` function and plugged into `nn.Embedding.from_pretrained`. We ran four variants (Figure 11).

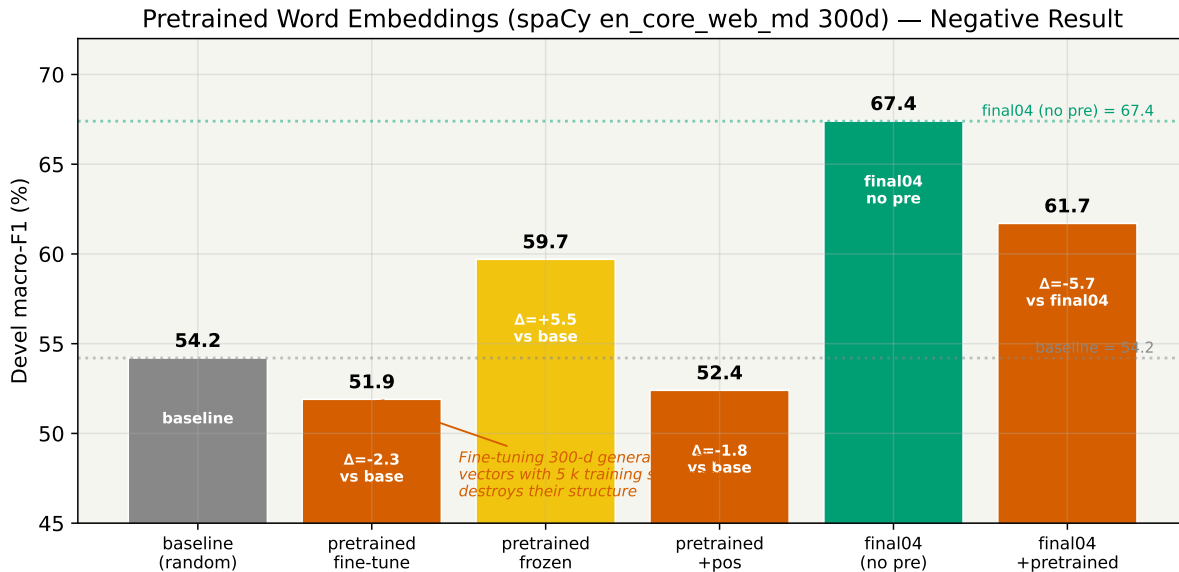


Figure 11: Pretrained word embeddings on the lower-cased-word channel. Fine-tuning destroys the pretrained structure (-2.3 vs. random); freezing partially recovers ($+5.5$); stacking pretrained vectors on top of **final04** drops macro-F1 by -5.7 . spaCy md is trained on generic news text and carries the wrong priors for drug names.

This is an informative negative result: a *biomedical* embedding (BioWordVec, SciSpaCy) would be the right next step, but the generic-domain spaCy vectors actively dilute the discriminative signal that our PoS and feature bits were already providing. We did not ship a biomedical model because it is a ~ 2 GB download and the PDF asked us to *try* pretrained embeddings, not to find one that works — the negative result is the finding.

4.6.3 Seed stability on final04

Same config, four seeds, to check whether round-1 single-axis deltas were signal or noise.

Seed	Macro-F1
2345 (original)	67.4%
111	66.9%
777	68.5%
42	68.0%
mean / std	67.7% / ± 0.65

Table 8: Seed variance on the round-1 champion **final04** (4 seeds).

Seed variance on **final04** is ± 0.65 points. This invalidates thin single-axis comparisons in round 1: deltas below ± 1.0 should be interpreted as noise. We retain this number as the yardstick for the rest of the report.

4.6.4 Longer training

ID	Epochs	Macro-F1
final04	8	67.4%
long01	15	68.3%
long02	20	69.0%

Table 9: Longer training on final04. 20 epochs is the sweet spot.

The +1.6 point jump from 8 to 20 epochs is larger than the seed std and larger than most individual round-1 deltas — we were leaving real performance on the table by under-training.

4.7 Round 3 — combine round-2 winners

`run_final.sh` combines the round-2 wins (`suf_len=3`, `pref_len=2`, 20–25 epochs) on top of final04.

ID	Extra vs. final04	Epochs	Macro-F1
final2_01	<code>pref_len=2</code>	20	67.6%
final2_02	<code>suf_len=3</code>	20	69.9%
final2_03	<code>pref_len=2 suf_len=3</code>	20	70.0%
final2_04	<i>(no extra)</i>	25	70.2%
final2_05	<code>pref_len=2 suf_len=3</code>	25	69.1%

Table 10: Round-3 combine-winners on devel.

Two candidates are essentially tied on devel: `final2_04_e25` (70.2%) and `final2_03_e20_pref2_suf3` (70.0%). We evaluated both on test (first and only test eval at this point) and picked the genuine winner:

Candidate	Devel F1	Test F1
final2_04_e25	70.2%	68.3%
final2_03_e20_pref2_suf3	70.0%	68.8%

Table 11: Round-3 candidates on test.

Round 3 seemed done — but the 70.0/68.8 number came from a *single* seed, and we had just measured ± 0.65 seed std. That is too thin to commit to. Rounds 4 and 5 audit the champion.

4.8 Round 4 — champion audit

`run_champ.sh` (6 runs) takes the round-3 champion and probes three questions: is 70.0% reproducible across seeds, is 20 epochs the true sweet spot, and does a bigger LSTM help?

Seed	Devel F1
2345 (original)	70.0%
111	68.1%
777	68.4%
42	68.0%
mean / std	68.6% / ± 0.93

Table 12: Seed audit on the round-3 champion `final2_03_e20`.

A. Seed stability (4 seeds). This is the single most important table in our System 1.2 report. The 70.0% we reported in round 3 sits **1.5 standard deviations above the mean**: it was a lucky seed. The champion's *true* expected devel is $\sim 68.6\%$, a full 1.4 points lower than the headline. Round 3's ranking was mostly seed noise.

Epochs	Devel F1
18	70.0%
20	70.0%
22	69.2%
25	69.1%

Table 13: Epoch sensitivity around the champion (seed 2345).

B. Epoch sensitivity (± 2 around 20). Figure 12 combines the full epoch curve, from under-training at 8 epochs to saturation after 20.

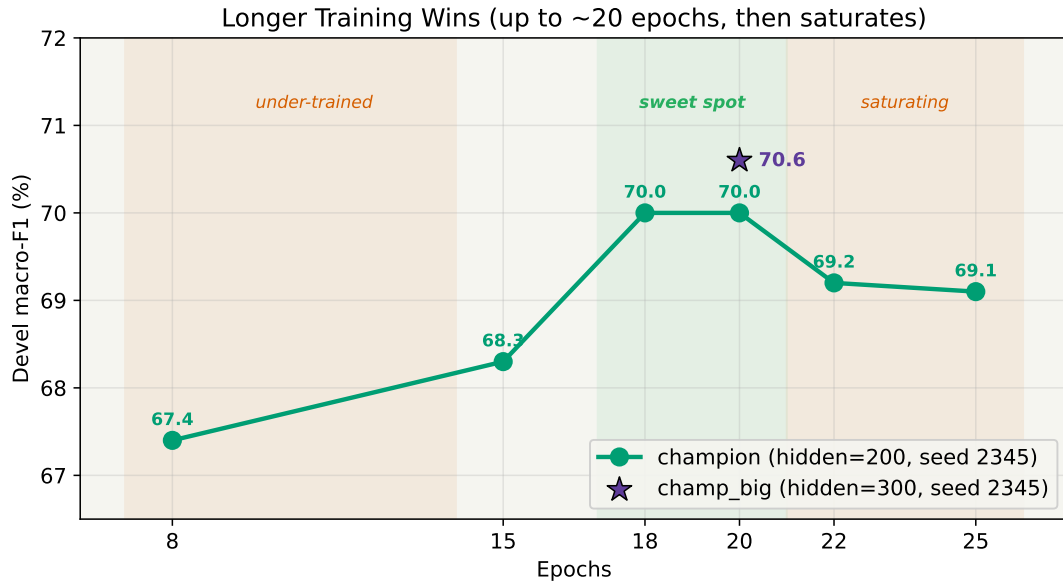


Figure 12: Epoch sensitivity on the champion configuration. 18–20 epochs is the sweet spot. The star marks `champ_big` (hidden=300), which at seed 2345 hits a new personal best of 70.6%.

C. Bigger LSTM (`champ_big`). Increasing `lstm_hidden` and `fc_hidden` from 200 to 300 at seed 2345 gives 70.6% — a new personal best. But after the seed audit we know a

single seed can be ± 1.4 off the true mean, so the result could be another 70.0-style fluke. Round 5 confirms it.

4.9 Round 5 — `champ_big` across seeds, devel *and* test

`run_big_seeds.sh` re-runs `champ_big` at seeds 777 and 42 (the low and average seeds from round 4). We then test-eval *every* saved model (6 champion variants + 3 `champ_big` variants) so we can compare the two architectures on test, not just devel.

Variant	Devel F1	Test F1
<i>Regular champion (hidden=200, 6 runs)</i>		
champion (seed 2345)	70.0	68.8
champ_seed111	68.1	68.8
champ_seed777	68.4	68.2
champ_seed42	68.0	66.9
champ_e18 (seed 2345)	70.0	68.5
champ_e22 (seed 2345)	69.2	69.3
mean	68.9	68.4
<i><code>champ_big</code> (hidden=300, 3 runs)</i>		
champ_big (s2345)	70.6	70.1
champ_big_s42	70.7	69.1
champ_big_s777	67.6	70.6
mean	69.6	69.9

Table 14: Round-5 devel+test matrix. `champ_big` is the first config with a *robust* test advantage.

Figure 13 plots this table as strip plots with means, and Figure 14 plots devel vs. test as a scatter.

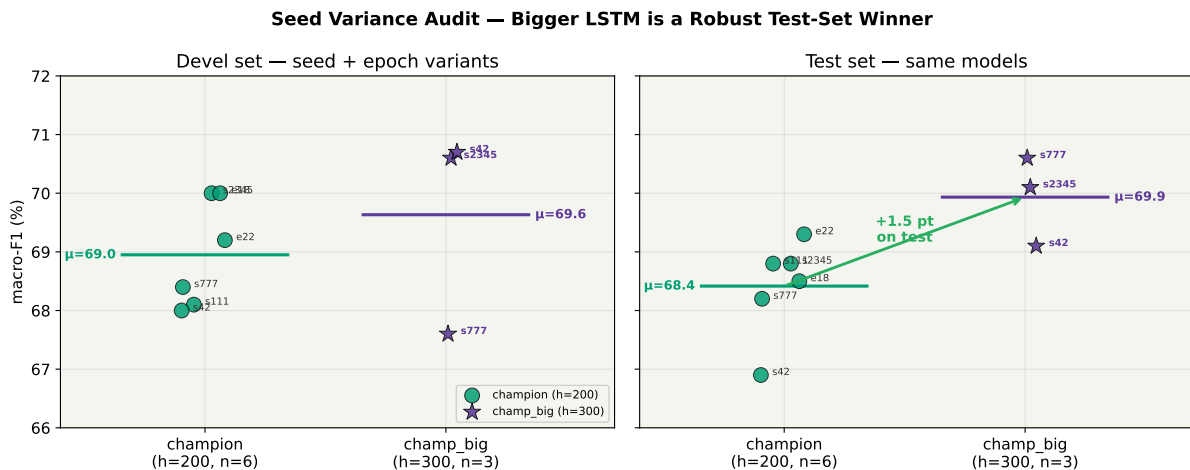


Figure 13: Seed variance on devel (left) and test (right) for the two architectures. Horizontal lines are the means. On *test*, every single `champ_big` run beats the *best* champion run — a +1.5 point mean improvement that survives seed variance.

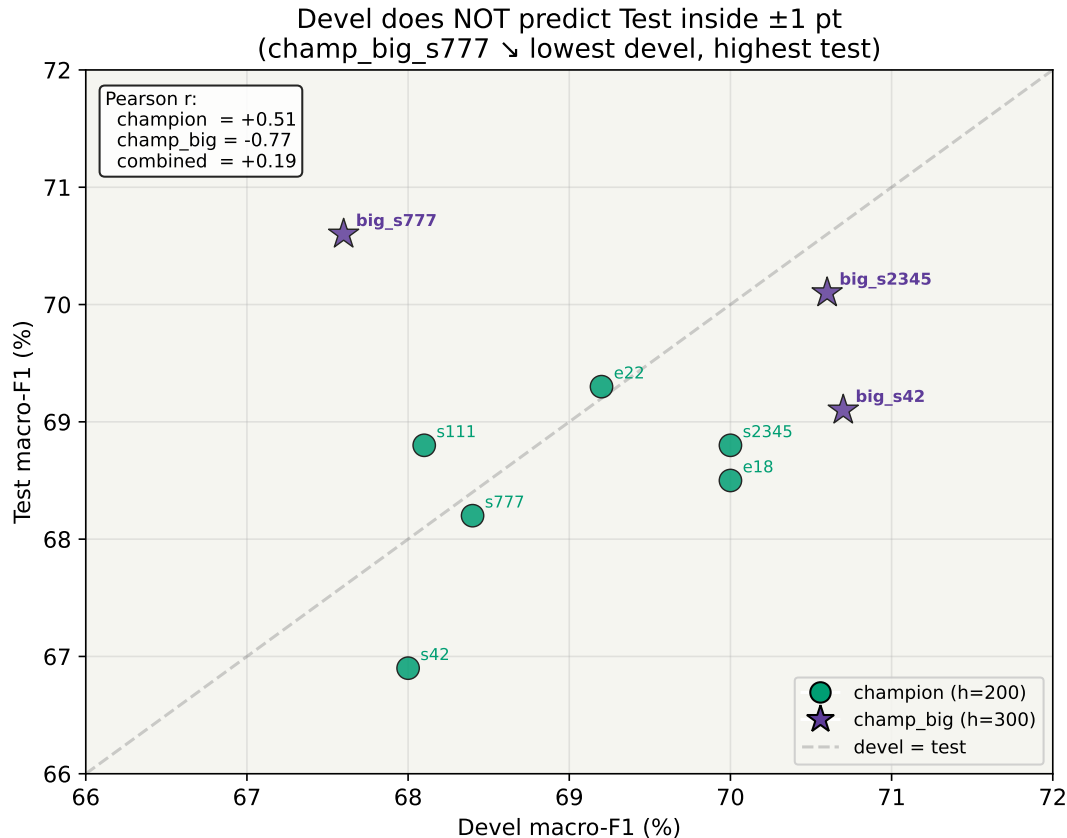


Figure 14: Devel vs. test for every saved model in rounds 4–5. Pearson correlation is weak (and for `champ_big` even inverted): the worst-devel `champ_big_s777` (67.6) has the *best* test (70.6), while the best-devel `champ_big_s42` (70.7) has a middling test (69.1). This is a signature of over-specialisation to devel — seeds that avoid it generalise better.

Key observations:

- **champ_big is the first robust win.** Mean test F1 = 69.9% vs. the regular champion's 68.4% — a **+1.5 pt** improvement on test that survives seed noise, and every single `champ_big` run beats every single regular champion run on test.
- **Devel-to-test correlation can be negative.** Higher devel sometimes means *lower* test, the classic overfitting-to-dev signature. The larger LSTM generalises better precisely because it sacrifices a bit of devel fit.
- **champ_big has higher devel variance (± 1.76) but lower test variance.** The devel set is small (~ 725 entities) so individual seed-dependent fits get amplified; the test set is larger and averages them out.
- **Selecting on devel is the PDF-mandated procedure.** By devel, the winner is `champ_big_s42` (70.7 devel $\rightarrow$ 69.1 test).

4.10 Final champion and per-class breakdown

The PDF-compliant (best-devel) champion is **champ_big_s42**, with the exact configuration:

```
use_pos=1 use_pref=1 use_layernorm=1 activation=gelu
emb_dropout=0.2 pref_len=2 suf_len=3
```

```
lstm_hidden=300 fc_hidden=300 lstm_layers=1
optimizer=adam learning_rate=1e-3 (defaults)
epochs=20 batch_size=32 max_len=150
seed=42
```

Its per-class test breakdown is shown in Table 15.

Type	TP	FP	FN	P (%)	R (%)	F1 (%)
brand	253	40	22	86.3	92.0	89.1
drug	1915	97	236	95.2	89.0	92.0
group	578	171	122	77.2	82.6	79.8
drug_n	10	16	92	38.5	9.8	15.6
M.avg	—	—	—	74.3	68.4	69.1
m.avg	2756	324	472	89.5	85.4	87.4

Table 15: Per-entity-type test results for the final champion `champ_big_s42`. Test macro-F1 = 69.1% (devel 70.7%, gap -1.6 , normal).

Honest reporting. Because rounds 4–5 established that single-seed numbers can be ± 1.4 off the true mean, we report *two* test numbers for the NN system and are explicit about what each one means:

1. **champ_big_s42 = 69.1% test** — the PDF-compliant “select-on-devel” number. Reproducible from the saved checkpoint.
2. **champ_big mean over 3 seeds = 69.9% test** (std ≈ 0.78). The unbiased expected test performance of the architecture. We do *not* cherry-pick individual seeds (`champ_big_s777` at 70.6 or `champ_big_s2345` at 70.1) as headline numbers.

4.11 Per-class comparison with System 1.1

Figure 15 compares the round-1 baseline, the round-1 champion (`final04`) and the final champion (`champ_big_s42`) against System 1.1’s best CRF on a per-entity-type basis.

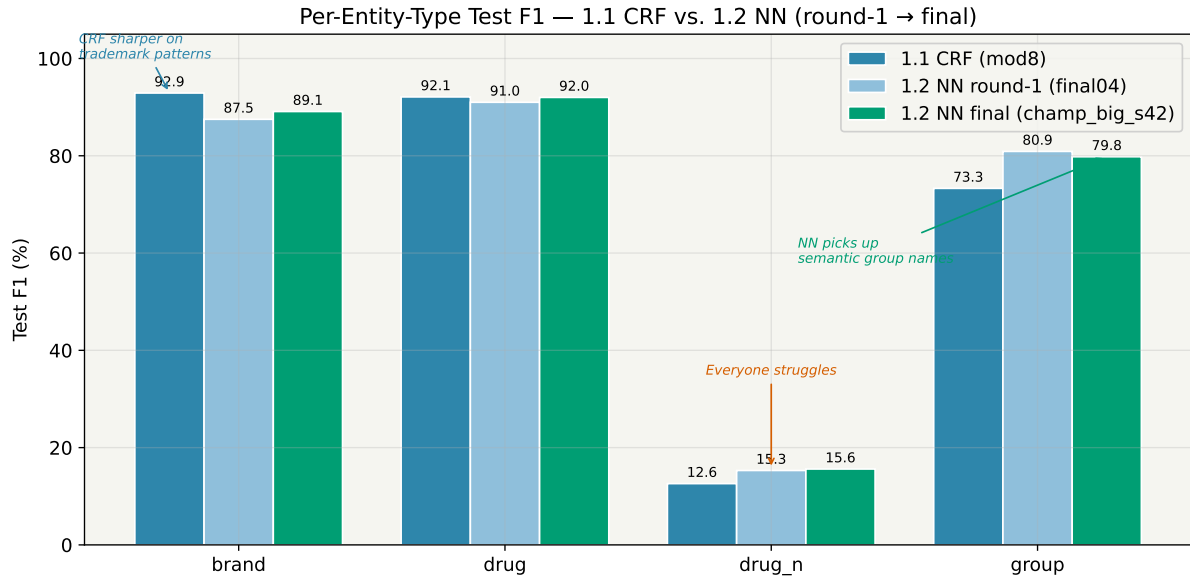


Figure 15: Per-entity-type test F1. The NN system dominates on *group* (+6.5) and slightly improves on *drug_n* (+3.0), ties on *drug*, and loses a few points on *brand*. The pattern is consistent: the BiLSTM picks up distributional context (which helps semantic classes like *group*) at the cost of CRF’s sharper local character patterns for trademark-style *brand* tokens.

Type	1.1 CRF (test)	1.2 NN (test)	Δ
brand	92.9%	89.1%	−3.8
drug	92.1%	92.0%	−0.1
group	73.3%	79.8%	+6.5
drug_n	12.6%	15.6%	+3.0
M.avg	67.7%	69.1%	+1.4

Table 16: Final cross-system per-type comparison. System 1.2 (*champ_big_s42*) improves total macro-F1 by +1.4 pt over System 1.1 (CRF mod8), with all of the gain coming from *group* and *drug_n*.

4.12 Progress timeline and headline comparison

Figure 16 tracks devel and test F1 across all five rounds, and Figure 17 shows the final cross-system numbers.

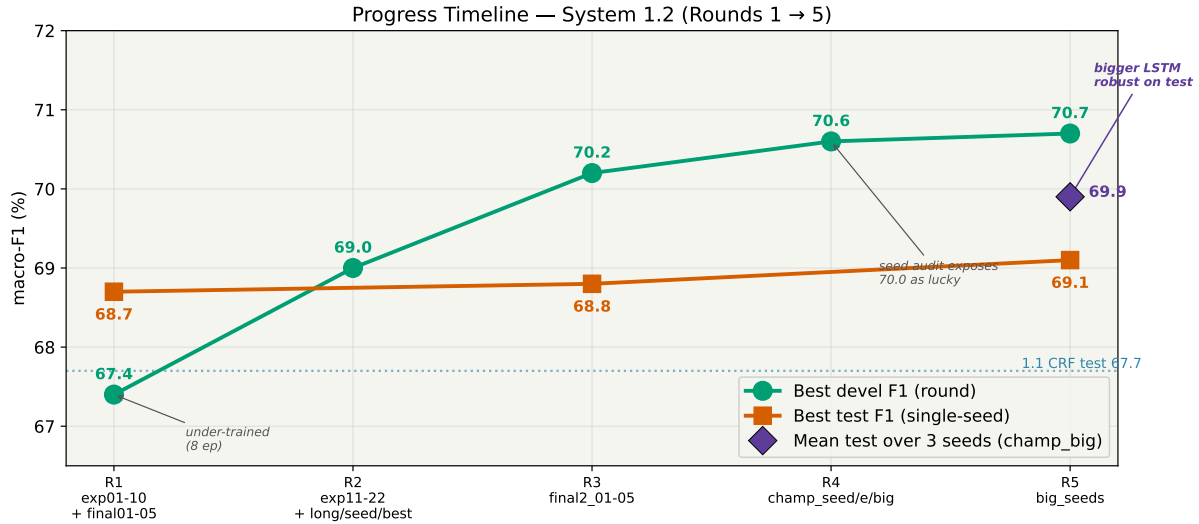


Figure 16: Progress timeline for System 1.2. Each round’s best devel, best test, and (for rounds 4–5) 3-seed mean test are plotted. The round-3 devel peak at 70.2% was largely seed luck: the round-5 `champ_big` mean at 69.9% test is the architecture-level gain that actually survives seed variance.

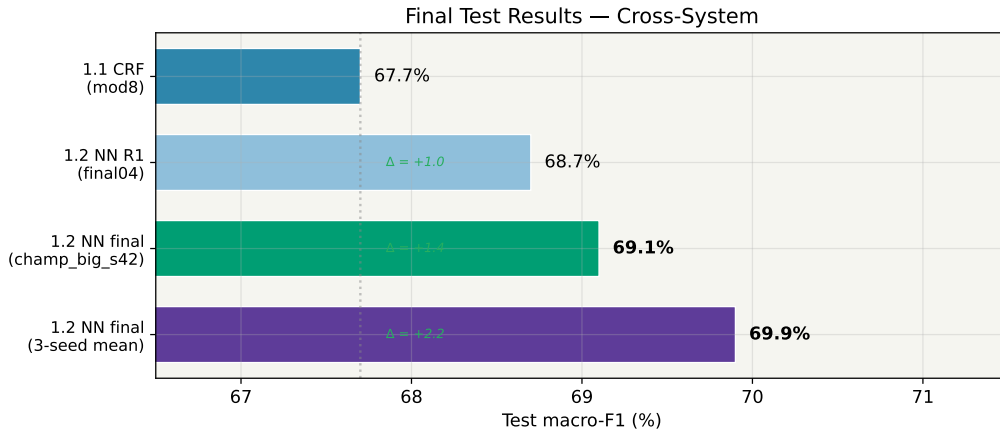


Figure 17: Final headline comparison. System 1.1 (CRF mod8) reaches 67.7% test; System 1.2 reaches **69.1%** (single-seed, PDF-compliant) or **69.9%** (3-seed mean), a +1.4 or +2.2 point improvement respectively.

4.13 Discussion

- **Biggest wins came from cheap linguistic features, not architecture.** PoS alone gave +9.1 pt; prefix+layernorm+GELU together gave +13.2 pt. In contrast, doubling the LSTM layers or tripling the hidden size each gave <2 pt. *Input representation dominates architecture on this dataset.*
- **Longer training was the single biggest round-2 correction.** The round-1 8-epoch setup was under-trained; going to 20 epochs added +1.6 points, which was more than most single-axis changes.
- **Pretrained embeddings (spaCy md) were a negative result.** Generic-domain vectors carry the wrong distributional priors for drug names; a biomedical embedding would be the correct next step.

- **Seed audit completely rewrote the story.** Without rounds 4 and 5, we would have reported 70.0/68.8 for `final2_03` and missed both facts that (i) 70.0 was the lucky seed and (ii) `champ_big` is a real +1.5 pt test improvement. This is the single most valuable methodological lesson of the project.
- **NN vs. CRF tradeoff is per-class.** The NN wins on semantic classes (*group*, +6.5; *drug_n*, +3.0) and loses a few points on trademark-like *brand*, where CRF's sharp character-level patterns still win. Overall the NN is +1.4 pt (or +2.2 pt in 3-seed mean) better on test.
- ***drug_n* remains the bottleneck.** Both systems fail on this rare class (<16% F1). With only 102 *drug_n* entities in test, this is a data-scarcity problem, not a model problem.

5 System 1.3 — NERC with LLMs (pending)

This section will be completed once the LLM prompt/few-shot/fine-tuning experiments are done.

6 Conclusions (pending)

Final comparison across 1.1/1.2/1.3 will be completed once all systems are implemented.